Univerzita Karlova

Matematicko-fyzikální fakulta

DOKUMENTACE K ZÁPOČTOVÉMU PROGRAMU

Název projektu:

Emulátor procesoru RISC-V (RV32I)

Předmět: Programování 1 (NPRG030)

Semestr: Zimní 2025/2026

Autor: Adam Jiřík

Obor: Informatika

Email: adam.jirik605@student.cuni.cz

Obsah

1. Zadání úlohy 3
2. Uživatelská příručka 3
3. Programátorská dokumentace 5
   3.1. Implementace paměťového systému (Třída `Memory`) 5
   3.2. Implementace procesoru (Třída `CPU`) 7
4. Závěr 13
5. Zdroje 14

1. Zadání úlohy

Cílem projektu je vytvoření funkčního emulátoru základní instrukční sady **RV32I** (architektura RISC-V). Emulátor musí simulovat:

1. **Adresovatelnou paměť** po bajtech s podporou Little Endian.
2. **Registrové pole** (32 univerzálních registrů x0–x31).
3. **Instrukční cyklus** Fetch-Decode-Execute pro základní aritmetické, logické, paměťové a skokové instrukce.
4. **Diagnostické nástroje** pro sledování vnitřního stavu procesoru během výpočtu (trace).

2. Uživatelská příručka

Program je navržen jako interaktivní konzolová aplikace. Pro jeho běh není potřeba instalovat žádné externí knihovny, stačí standardní prostředí jazyka Python 3. Po spuštění souboru se uživateli zobrazí hlavní menu.

<u>Podporované režimy:</u>

1. Volby 1-4: Interaktivní kalkulačka
   - Použití: Po zadání čísla odpovídající operacím (viz. Tabulka níže) vás program vyzve k zadání dvou celých čísel (operandů A a B). Program využívá robustní parsování vstupu pomocí Hornerova schématu. Uživatel může zadat číslo i v rámci textového řetězce (např. `A = 10jablek`). Program automaticky vyfiltruje *první* číselnou hodnotu (v tomto příkladě hodnotu `10`). Po potvrzení klávesou Enter je zbytek znaků na řádku zahozen. Pro zadání druhého operandu má uživatel k dispozici nový, čistí vstup.
   - Co program udělá: Emulátor čísla zpracuje a na obrazovku vypíše detailní záznam (trace) toho, jak by tento výpočet provedl skutečný procesor krok za krokem.
   - Výstup: Na konci záznamu program předhledně zobrazí konečnou operaci s výsledkem matematické operace.

2. Volba 5: Fibonacciho posloupnost (Demo)
   - Použití: Po zadání čísla 5 program automaticky nahraje do paměti předpřipravený strojový kód.
   - Co program udělá: Kód demonstruje složitější chování procesoru včetně cyklů a podmíněných skoků (instrukce BEQ/BNE). Vypočítá 7. prvek Fibonacciho posloupnosti.
   - Výstup: Uživatel vidí detailní provádění (trace) každé instrukce. Na konci se vypíše finální hodnota 7. prvku Fibonacciho posloupnosti.

3. Volba 6: Automatické testy
   - Použití: Zadáním čísla 6
   - Co program udělá: Spustí integrovanou sadu unit testů, které ověří správnost fungování paměti (včetně Little Endian zápisu) a aritmeticko-logické jednotky procesoru.

<u>Tabulka operací</u>

Volba	**Operace**	**Popis operace**
1	Sčítání	Sčítání dvou celočíselných operandů.
2	Odečítání	Odečítání dvou celočíselných operandů.
3	bit. AND	Provede logický součin (AND) na bitech.
4	bit. OR	Provede logický součet (OR) na bitech.
5	Fibonacci test	Spustí předpřipravený ukázkový program, který vypočítá 7. člen Fibonacciho posloupnosti. Nevyžaduje žádné inputy. Pouze ukázka.
6	Testy	Spustí automatickou vnitřní kontrolu celého emulátoru. Slouží k rychlému ověření, že program funguje správně a bez chyb.
0	Konec programu	Ukončí aplikaci.

3. Programátorská dokumentace

Základní architektura našeho emulátoru je rozdělena na dvě oddělené komponenty: paměťový systém `Memory` a samostatný procesor `CPU`. Tento přístup, kdy třída `CPU` přijímá v konstruktoru instanci třídy `Memory` zajišťuje správné rozdělení odpovědnosti. Procesor se tedy nemusí vůbec zajímat o to, jak a kde jsou data v paměti fyzicky uložena.

3.1. Implementace paměťového systému (Třída `Memory`)

Třída `Memory` je implementována jako pole, kde každý index reprezentuje jeden fyzický byte. Hodnoty uvnitř pole jsou omezeny na rozsah 0-255, což odpovídá formátu *unsigned 8-bit integer*. Výchozí velikost paměti je alokována na 1 MiB (1024 x 1024 bajtů), což je pro běh našich testovacích a demonstračních algoritmů plně dostačující. Součástí paměti je i kontrola mezí (boudns checking) simulující hardwarovou výjimku *Segmentation Fault* při pokusu o přístup mimo alokovaný prostor.

<u>Little Endian a skládání slov</u>

Zatímco adresujeme po bytech, samotný procesor zpracovává instrukce a data primárně po blocích o velikosti 32 bitů (tzv. slovo), což odpovídá 4 bytům. Musíme tedy vyřešit, v jakém pořadí se tyto 4 byty budou ukládat. Naše implementace využívá Little Endian pořadí.

- Princip: Nejméně významný byte (Least Significant Byte – LSB) je uložen na nejnižší adrese, zatímco nejvíce významný byte je uložen na nejvyšší adrese. Metody `read_32` a `write_32` se nám starají o to, aby se z 32-bitového celku (slova) byty vyřízly (pomocí bitových posunů `>>` a masky `& 0xFF`) a uložily se na správné adresy dle Little Endian pořadí.

Konkrétní příklad ukládání do paměti:

Uložme 32 bitové hexadecimální číslo 0x12345678.

- Adresa 0: 0x78 (LSB)
- Adresa 1: 0x56
- Adresa 2: 0x34
- Adresa 3: 0x12 (MSB)

3.2. Implementace procesoru (Třída `CPU`)

Jádro emulátoru se nachází ve tříde CPU, která funguje jako mozek počítače. Zde provádíme instrukční cyklus *Fetch-Decode-Execute*.

Zpracování uživatelského vstupu

Vzhledem k nízkoúrovňové verzi projektu byla pro načítání číselných vstupů namísto standardní funkce `int()` implementována vlastní funkce `get_num()`. Tato funkce pracuje přímo s přímým vstupem (`sys.stdin`):

- Filtrování nečíselných znaků: Funkce čte vstup po jednotlivých znacích. Veškeré znaky, které nenáleží do ASCII rozhasu ‘0’ – ‘9’ (48 – 57) jsou ignorovány, dokud není nalezen první číselný znak.
- Využití Hornerova schématu: Nalezená sekvence číslic je převáděna na celočíselnou hodnotu (integer) pomocí algoritmu Hornerova schematu:

  výsledek = výsledek * 10 + (ascii_znak – ‘0’)

- Ukončení a čištění bufferu: Parsování čísla končí narazením na první nečíselný znak po začátku sekvence číslic.

Registry

Procesor obsahuje 32 univerzálních registrů (General Purpose Registers), označených `x0` až `x31`. Do každého z nich se vejde jedno 32-bitové číslo. Pro diagnostiku je navíc implementována metoda `dump_registers()`, která dokáže vypsat stav registrů v desítkové soustavě.

Speciální registr: `x0` – Registr `x0` je v architektuře RISC-V speciální, protože je hardwarově uzemněn (hardwired) na hodnotu 0 a jakýkoliv pokud o zápis do něj se ignoruje. V emulátoru je to pojištěno měkkou (softwarově) podmínkou v metodě `set_reg()`.

Program Counter

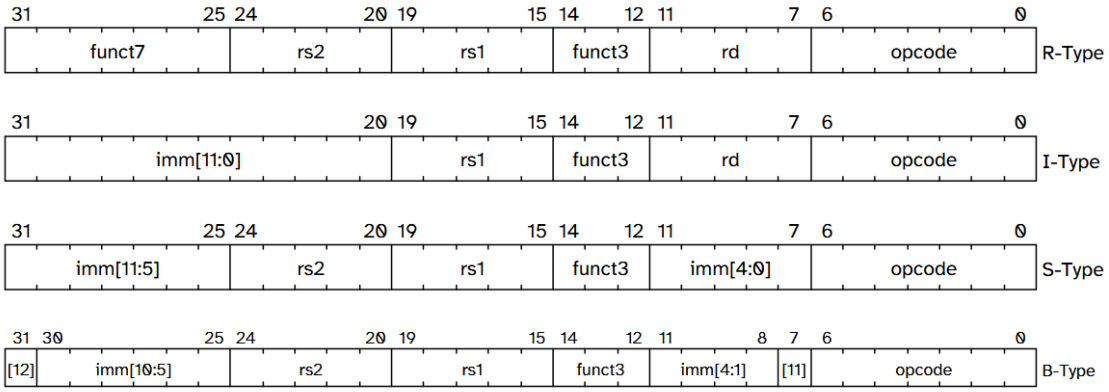
Speciální 32-bitový registr nedrží data pro výpočty, ale funguje jako ukazatel (pointer) na aktuální adresu vykonávané instrukce. Po resetu emulátoru se vždy resetuje na hodnotu `0x00000000`. Jelikož má každá instrukce architektury RISC-V délku 32 bitů (4 byty), hodnota PC se po úspěšném vykonání instrukce standardně zvyšuje o 4. Výjimkou jsou skokové instrukce (např. BNE, BEQ), které k PC přičítají relativní offset, čímž dochází k větvení programu.

Instrukční cyklus

Samotný běh procesoru je řízen smyčkou (metoda `step()`), která v každém taktu provádí tři základní kroky:

1. Fetch (Načtení): Procesor přečte z paměti 32-bitové slovo na adrese, na kterou aktuálně ukazuje Program Counter. Pokud by PC ukazoval mimo alokovanou paměť, smyčka se ukončí.
2. Decode (Dekódování): Přečtená instrukce je rozložena na jednotlivé části. Protože RISC-V používá pevný formát instrukcí, využíváme extrakci bitových polí, bitové posuny a masky. Například 7-bitový operační kód (opcode) získáme aplikací masky `& 0x7F` na nejnižší bity instrukce.
3. Execute (Vykonání): Na základě dekódovaného operačního kódu a rozšiřujících bitů (např. `funct3`, `funct7`) program identifikuje konkrétní operaci a provede ji (např. aritmetický součet, přístup do paměti či vyhodnocení podmínky pro skok). Na konci této fáze se aktualizuje hodnoty PC a změny se zapíší do registrů či do paměti.

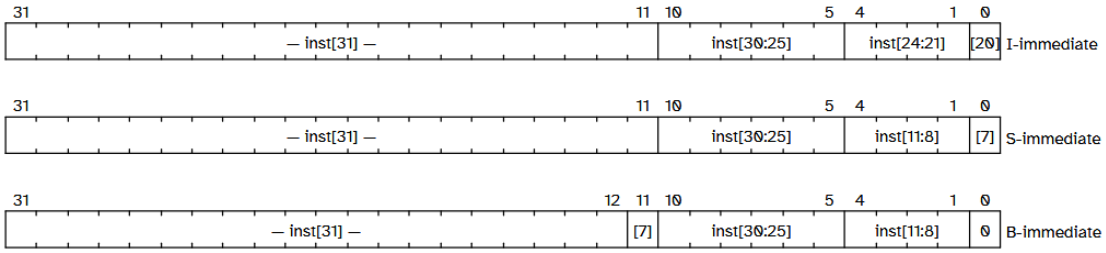
Instrukce	**Typ**	**Opcode**	**Funct3**	**Funct7**	**Popis**
ADD	R	0x33	0x0	0x00	Sčítání dvou registrů
SUB	R	0x33	0x0	0x20	Odčítání dvou registrů
AND	R	0x33	0x7	0x00	Logický součin nad registry
OR	R	0x33	0x6	0x00	Logický součet nad registry
XOR	R	0x33	0x4	0x00	Exkluvizní logický součet
ADDI	I	0x13	0x0	-	Sčítání s konstantou
ANDI	I	0x13	0x7	-	Logický součin s konstantou
LW	I	0x03	0x2	-	Načtení 32-bitového slova z paměti
SW	S	0x23	0x2	-	Zápis 32-bitového slova z paměti
BEQ	B	0x63	0x0	-	Skok, pokud jsou hodnoty registrů shodné
BNE	B	0x63	0x1	-	Sko, pokud se hodnoty registrů liší



Obrázek 1: Základní formáty instrukcí architektury RV32I. Převzato z [1, s. 26]

Zpracování konstant (Immediates)

Konstanty (immediates) jsou v instrukcích RISC-V zakódovány specifických způsobem napříč různými bity. Emulátor implementuje jejich skládání pomocí bitových operací pro formáty I-Type (slouží pro aritmetiku a načítání z paměti), S-type (slouží pro zápis do paměti) a B-type (pro podmíněné skoky). Zvláštní pozornost je věnována tzv. znaménkovému rozšíření (sign-extension) pro záporné hodnoty, které je ošetřeno manuálním odečtením příslušné mocniny dvojky v případě, že je nejvyšší (znaménkový) bit nastaven na 1.



Obrázek 2: Schéma skládání konstant (immediates) pro jednotlivé formáty instrukcí. Převzato z [1, s. 27]

Testování

Pro ověření funkčnosti emulátoru je využita standardní knihovna `unittest`. Architektura s oddělenou pamětí a procesorem umožňuje psát izolované testy pro jednotlivé komponenty. Mezi hlavní testované scénáře patří:

1. Testování paměti (`test_memory`): Ověřuje správnost skládání a rozkládání 32-bitových slov na bajty a jejich zápis na správné adresy podle zápisu Little endian.
2. Aritmetické operace (`test_cpu_ops`): Testuje Základní instrukce (ADDI, ADD) a kontroluje, zda procesor správně manipuluje s registry a ukládá očekávaný výsledek.
3. Řízení toku programu (`test_branching`): Testuje funkčnost podmíněných skoků (BNE, BEQ) a ověřuje, zda procesor správně vyhodnocuje shodu operandů a přesně upravuje PC (Program Counter)

Příklad chodu programu:

Sčítání čísel 5, 10.

```
========================================
RISC-V EMULATOR
========================================

Vyberte operaci:
1: Scitani (ADD)
2: Odcitani (SUB)
3: AND
4: OR
5: Fibonacci
6: Testy
0: Konec

Vase volba: 1
Zadejte prvni cislo (A): 5
Zadejte prvni cislo (B): 10



--- START VYPOCETNIHO CYKLU ---

[FETCH]  Adresa: 0x0000 | Instrukce: 0x002081B3
[EXEC]   ADD: x3 = x1(5) + x2(10)
[WRITE]  Ukladani hodnoty 15 do registru x3

  --- KONEC VYPOCETNIHO CYKLU ---

Vysledek: 5 + 10 = 15
```

4. Závěr

Cílem tohoto zápočtového programu bylo vytvořit plně funkční podmnožinu emulátoru procesoru architektury RISC-V. Podařilo se mi naimplementovat lineární paměťový model s přesným adresováním po bytech i kompletní instrukční cyklus, který zvládá základní artimetiku, logické operace, práce s pamětí a podmíněné větvění. Správnost fungování byla demonstrována na komplexnějším výpočtu Fibonacciho posloupnosti a ověřena sadou automatizovaných testů.

Možným rozšířením do budoucna by mohla být kompletní implementace celé množiny architektury, případně podpora nahrávání zkompilovaných binárních souborů přímo z kompilátoru, čímž by se emulátor stal využitelnějším pro běh složitějších reálných programů.

Při programování mi dalo největší práci se zorientovat v hardwarové specifikaci RISC-V a pochopit, jak se pomocí bitových operací skládají konstanty (immediates) instrukcí. Zjistil jsem, jaký rozdíl me mezi psaním kódu ve vyšších programovacích jazycích a tím, jak data reálně zpracovává procesor na úrovni jednotlivých bitů a bytů. Zpětně musím říct, že i přes počáteční zmatky s oficiálním manuálem mi tvorba emulátoru velmi pomohla pochopit, co se skutečně dětě „pod kapotou“ počítače. Celkově pro mě ale byl tento projekt obrovskou zkušeností.

5. Zdroje

[1] RISC-V INTERNATIONAL. *The RISC-V Instruction Set Manual Volume I: Unprivileged ISA*. Online. Version 20250508. RISC-V International, 2019. Dostupné z: https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf. [cit. 2026-02-25].